

Chapter 11: Meta-analysis and measurement error models

Shravan Vasishth

2026-03-29

Contents

<i>1</i>	<i>Introduction</i>	<i>2</i>
<i>2</i>	<i>Meta-analysis</i>	<i>3</i>
<i>2.1</i>	<i>A meta-analysis of similarity-based interference in sentence comprehension</i>	<i>3</i>
<i>2.2</i>	<i>brms version of the meta-analysis model</i>	<i>5</i>
<i>2.3</i>	<i>Meta-regression</i>	<i>7</i>
<i>2.4</i>	<i>Aside: Frequentist version of the random-effect meta-analysis</i>	<i>10</i>
<i>2.5</i>	<i>Aside: What happens if we set <code>sigma = TRUE</code> in <code>brms</code>?</i>	<i>12</i>
<i>2.6</i>	<i>Back to meta-analysis: Forest plots</i>	<i>14</i>
<i>2.7</i>	<i>Stan version of the meta-analysis model</i>	<i>16</i>
<i>3</i>	<i>Measurement-error models</i>	<i>21</i>
<i>3.1</i>	<i>Accounting for measurement error in individual differences in working memory capacity and reading fluency</i>	<i>21</i>
<i>3.2</i>	<i>The <code>brms</code> version of the measurement error model</i>	<i>26</i>
<i>3.3</i>	<i>The <code>Stan</code> version of the measurement error model</i>	<i>29</i>

Please also read chapter 13 of the textbook.

1 Introduction

In this lecture, I introduce two relatively underutilized (in psychology and linguistics) models that are potentially very important for cognitive science: meta-analysis and measurement-error models. These topics are also related to the hierarchical modeling introduced in chapter 5.

Meta-analysis is commonly used in medicine; see for example:

Chandler, J., Cumpston, M., Li, T., Page, M. J., & Welch, V. J. H. W. (2019). *Cochrane handbook for systematic reviews of interventions*. Hoboken: Wiley.

Meta-analysis can be very informative when carrying out systematic reviews, and measurement-error models are able to take into account uncertainty in one's dependent or independent variable (or both). What's common to these two classes of model is that they both assume that the n -th measured data point y_n has an unknown true value of a parameter, say ζ_n (pronounced *zeta en*), that is measured with some uncertainty that can be represented by the standard error SE_n of the measurement y_n :

$$y_n \sim \text{Normal}(\zeta_n, SE_n) \quad (1)$$

In both classes of model, the goal is to obtain a posterior distribution of a **latent parameter** ζ which is assumed to generate the ζ_n , with some standard deviation τ . The parameter τ quantifies the noise in the measurement process or the between-study variability in a meta-analysis.

$$\zeta_n \sim \text{Normal}(\zeta, \tau) \quad (2)$$

The main parameter of interest is usually ζ , but the posterior distributions of τ and ζ_n can also be informative. The above model specification should remind you of the hierarchical models we saw in earlier chapters, because this is exactly a varying intercepts model. There is an overall effect ζ , and individual studies' estimates ζ_n vary about that overall effect, with standard deviation τ .

2 Meta-analysis

Once a number of studies have accumulated on a particular topic, it can be very informative to synthesize the data. Here is a commonly used approach—a random-effects meta-analysis. For other types of meta-analysis, see:

Chen, D. G. D., & Peace, K. E. (2013). *Applied meta-analysis with R*. CRC Press.

2.1 A meta-analysis of similarity-based interference in sentence comprehension

The model is set up as follows. For each study n , let effect_n be the effect of interest, and let SE_n be the standard error of the effect. A concrete example of a recent meta-analysis is the effect of similarity-based interference in sentence comprehension is Jäger et al 2017;¹ when two nouns are more similar to each other, there is greater processing difficulty (i.e., longer reading times in milliseconds) when an attempt is made to retrieve one of the nouns to complete a linguistic dependency (such as a subject-verb dependency). The estimate of the effect and its standard error is the information we have from each study n .

First, load the data, and add an id variable that identifies each experiment.

```
data("df_sbi")
(df_sbi <- df_sbi %>%
  mutate(study_id = 1:n()))
```

```
## # A tibble: 12 x 4
```

##	publication	effect	SE	study_id
##	<chr>	<int>	<int>	<int>
## 1	VanDyke07E1LoSem	13	30	1
## 2	VanDyke07E2LoSem	37	21	2
## 3	VanDyke07E3LoSem	20	11	3
## 4	VanDykeEtal03E4	56	25	4
## 5	VanDykeEtal11E2bPro	7	9	5
## 6	VanDykeEtal11E2bRetro	-7	9	6
## 7	VanDyke07E3LoSyn	8	8	7
## 8	VanDyke07E1LoSyn	54	34	8
## 9	VanDyke07E2LoSyn	44	19	9
## 10	VanDykeEtal06	38	20	10
## 11	VanDykeEtal11E1bPro	5	8	11

¹ Jäger, L. A., Engelmann, F., & Vasishth, S. (2017). Similarity-based interference in sentence comprehension: Literature review and Bayesian meta-analysis. *Journal of Memory and Language*, 94, 316-339.

12 VanDykeEtAl11E1bRetro -2 11 12

The effect size and standard errors were estimated from published summary statistics in the respective article. In some cases, this involved a certain amount of guesswork because authors don't always provide enough information to obtain the essential estimates (effect size and standard error); the details are documented in the online material accompanying the 2017 paper.²

² See: <https://shorturl.at/RhQV5>

We begin with the assumption that there is a true (unknown) effect ζ_n that lies behind each of these studies. Each of the observed effects has an uncertainty associated with it, SE_n . We can therefore assume that each observed effect, $effect_n$, is generated as follows:

$$effect_n \sim Normal(\zeta_n, SE_n) \quad (3)$$

Each study is assumed to have a different true effect ζ_n because each study will have been carried out under different conditions: in a different lab with different protocols and workflows, with different subjects, with different languages, with slightly different experimental designs, etc.

Further, each of the true underlying effects ζ_n has behind it some true unknown value ζ . The parameter ζ represents the underlying effect of similarity-based interference across experiments. Our goal is to obtain the posterior distribution of this overall effect.

We can write the above statement as follows:

$$\zeta_n \sim Normal(\zeta, \tau) \quad (4)$$

τ is the between-study standard deviation; this expresses the assumption that there will be some variability between the true effects ζ_n .

To summarize the model:

- $effect_n$ is the observed effect (in this example, in milliseconds) in the n -th study.
- ζ_n is the true (unknown) effect in each study.
- ζ is the true (unknown) effect of the experimental manipulation, namely, the similarity-based interference effect.
- Each SE_n is estimated from the standard error available from study n .

- The parameter τ represents between-study standard deviation.

We can construct a hierarchical model as follows:

$$\begin{aligned} \text{effect}_n &\sim \text{Normal}(\zeta_n, SE_n) \quad n = 1, \dots, N_{\text{studies}} \\ \zeta_n &\sim \text{Normal}(\zeta, \tau) \\ \zeta &\sim \text{Normal}(0, 100) \\ \tau &\sim \text{Normal}_+(0, 100) \end{aligned} \tag{5}$$

The priors are based on domain knowledge; it seems reasonable to allow the effect to range a priori from -200 to $+200$ ms with probability 95%. Of course, a sensitivity analysis is necessary (but skipped here).

This model can be implemented in `brms` in a relatively straightforward way as shown below. We show the Stan version below; surprisingly for such a simple model, the Stan version presents some interesting challenges.

2.2 *brms version of the meta-analysis model*

First, define the priors:

```
priors <- c(prior(normal(0, 100), class = Intercept),
           prior(normal(0, 100), class = sd))
```

Fit the model as follows. Because of our relatively uninformative priors and the few data points, the models of this chapter require us to tune the `control` parameter, increasing `adapt_delta` and `max_treedepth`.

```
fit_sbi <- brm(effect | resp_se(`SE`, sigma = FALSE) ~ 1
              + (1 | study_id),
              data = df_sbi,
              prior = priors,
              control = list(
                adapt_delta = .99,
                max_treedepth = 10))
```

The posterior of ζ and τ are summarized below as `Intercept` and `sd(Intercept)`.

```
short_summary(fit_sbi)
```

```
## ...
## Group-Level Effects:
## ~study_id (Number of levels: 12)
##           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept)    11.43      7.67    0.52   28.53 1.00      782    1426
##
## Population-Level Effects:
##           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept     13.19      6.17    2.92   27.21 1.01     1311    1198
##
## Family Specific Parameters:
##           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sigma         0.00      0.00    0.00    0.00  NA        NA        NA
##
## ...
```

The sigma parameter does not play any role in this model, but appears in the brms output anyway. In the model specification, sigma was explicitly removed by writing `sigma = FALSE`. For this reason, we can ignore that parameter in the model summary output above. We explain below what happens if we set `sigma = TRUE`.

As theory predicts, the overall effect from these studies has a positive sign.

One advantage of such a meta-analysis is that the posterior can now be used as an informative prior for a future study. This is especially important when doing an analysis using Bayes factors. But this meta-analysis posterior could also be used as an informative prior in a future experiment; that would allow the researcher to build on what is known so far from published studies.

For examples from my lab using informative priors from meta-analyses for Bayes factors analyses, see

- Bruno Nicenboim, Shravan Vasishth, and Frank Rösler.
Are words pre-activated probabilistically during sentence comprehension? Evidence from new data and a Bayesian random-effects meta-analysis using publicly available data. *Neuropsychologia*, 142, 2020. <https://doi.org/10.1016/j.neuropsychologia.2020.107427>
- Pia Schoknecht, Himanshu Yadav, and Shravan Vasishth.
Do syntactic and semantic similarity lead to interference

effects? Evidence from self-paced reading and event-related potentials using German. *Journal of Memory and Language*, 2025. In Press. <https://osf.io/preprints/psyarxiv/cwymg/>

2.3 Meta-regression

If one also has experimental data that manipulated a predictor or multiple predictors, one can include these in the meta-analysis; this is called a meta-regression.

For example, if we include a predictor (continuous or factorial) in the model, one more parameter, β , is added into the model, which then requires a prior:

$$\begin{aligned} \text{effect}_n &\sim \text{Normal}(\zeta_n + \beta \times \text{predictor}_n, SE_n) \quad n = 1, \dots, N_{\text{studies}} \\ \zeta_n &\sim \text{Normal}(\zeta, \tau) \\ \beta &\sim \text{Normal}(0, 100) \\ \zeta &\sim \text{Normal}(0, 100) \\ \tau &\sim \text{Normal}_+(0, 100) \end{aligned} \tag{6}$$

Example: Meta-regression in agreement attraction

One research question that has been investigated in sentence processing is agreement attraction configurations like these:

- a. *The key to the cabinets are on the table.
- b. *The key to the cabinet are on the table.

In these ungrammatical sentences, the noun in the prepositional phrase (cabinet/cabinets) intervenes between the subject (key) and the auxiliary verb (are). The noun in the distractor is often called a distractor, and the subject is called the target. This terminology arises because, in order to understand the sentence, we need to connect the subject to the verb, so key is the target for making this connection. The noun cabinet/cabinets is considered in some sentence processing theories as a source of interference—the reader finds it difficult to figure out whether the subject is key or cabinet/cabinets; that’s why the noun cabinet/cabinets is called a distractor.

Simplifying slightly, a theoretical claim in the literature is that reading time the auxiliary verb in condition a is faster than in a because in condition a, the distractor and verb have the same number marking; this number match is claimed to

confuse readers and they end up thinking that sentence a is grammatical; this is called **agreement attraction**. The effect is expected to be negative.

When the syntactic configuration is as shown above, target-distractor-verb, we say that this is a **retroactive** configuration.

It is also possible to have a configuration like distractor-target-verb. Examples:

- c. *The cabinets that the key are for seem to be very old.
- d. *The cabinet that the key are for seem to be very old.

Here, the distractor (cabinets/cabinet) appears first, then the target (ket) and then the verb (are). This is called a **proactive** configuration.

There is a theoretical claim in the literature that attraction effect is affected by whether the syntactic configuration is retroactive or proactive: the effect is expected to be larger when the distractor is closer to the verb (i.e., the retroactive configuration should have a more negative effect sign than proactive configuration). We can test this with existing data in a meta-regression (the data are from the Jäger et al 2017 paper).

First, we load the data:

```
mismatchagr<-read.table("data/MismatchAgreement.txt",header=TRUE)
## means
round(with(mismatchagr,tapply(Effect,IntType,mean)))

##   pro retro
## -33   -17
```

Descriptively, the proactive configuration has a **larger** effect (more negative effect) than the retroactive configuration—this goes against theory.

Set up an appropriate contrast coding:

```
## the meaning of the +/- 0.5 contrast coding:
xtabs(~IntType+proretro,mismatchagr)

##           proretro
## IntType -0.5 0.5
##   pro      0    7
##   retro    6    0
```

This coding implies that a negative sign on the β parameter means that proactive configuration has a bigger (more negative effect).

Next, isolate the relevant columns:

Isolate the relevant columns:

```
mismatchagr<-mismatchagr[,c("Publication","Effect","SE","proretro")]
str(mismatchagr)
```

```
## 'data.frame': 13 obs. of 4 variables:
```

```
## $ Publication: chr "WagersEtAl09E2" "WagersEtAl09E4" "LagoEtAl15E1" "WagersEtAl09E5" ...
```

```
## $ Effect : int -51 -42 -40 -37 -36 -33 -31 -29 -22 -15 ...
```

```
## $ SE : int 23 17 14 16 18 23 29 14 11 7 ...
```

```
## $ proretro : num 0.5 -0.5 0.5 -0.5 0.5 0.5 0.5 -0.5 0.5 0.5 ...
```

Fit the model:

The results:

```
summary(fit_sbi_metareg)
```

```
## Family: gaussian
```

```
## Links: mu = identity
```

```
## Formula: Effect | resp_se(SE, sigma = FALSE) ~ 1 + proretro + (1 | Publication)
```

```
## Data: mismatchagr (Number of observations: 13)
```

```
## Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
```

```
## total post-warmup draws = 4000
```

```
##
```

```
## Multilevel Hyperparameters:
```

```
## ~Publication (Number of levels: 13)
```

```
## Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
```

```
## sd(Intercept) 16.36 7.31 2.32 33.06 1.00 1093 659
```

```
##
```

```
## Regression Coefficients:
```

```
## Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
```

```
## Intercept -21.62 6.84 -36.09 -9.12 1.00 1853 1882
```

```
## proretro -13.82 12.93 -39.28 12.02 1.00 2006 2240
```

```
##
```

```
## Further Distributional Parameters:
```

```
## Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
```

```
## sigma 0.00 0.00 0.00 0.00 NA NA NA
```

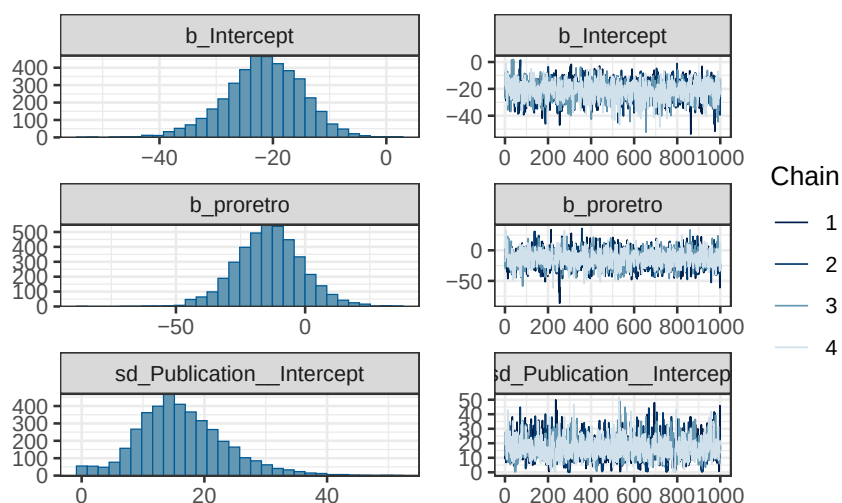
```
##
```

```
## Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
```

```
## and Tail_ESS are effective sample size measures, and Rhat is the potential
```

```
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(fit_sbi_metareg,variable=c("b_Intercept","b_proretro","sd_Publication__Intercept"))
```



We do see an overall attraction effect (the intercept, corresponding to ζ in the mathematical statement of the model), but the pro-/retroactive configuration seems to have the opposite effect relative to what theory predicts, given these data. This surprising result may be due to the sparsity of the data at hand; there is also a lot of between-study variability. Alternatively, the theoretical claim has been shown to be wrong, at least in these data.³

One could do a Bayes factor analysis to test the hypothesis formally. I will return to this point later in the course, as a formal Bayes factor analysis will give us more information about whether we can quantify the evidence for or against a theoretical prediction.

³ In fact, with more data, the expected pattern is seen. See the Jäger et al 2017 paper.

2.4 Aside: Frequentist version of the random-effect meta-analysis

We could have fit a frequentist meta-analysis:

```
## run the meta-analysis:
m <- rma(yi=effect,
        vi=SE^2,
        method = "REML",
        slab = df_sbi$study_id,
        data = df_sbi)
summary(m)

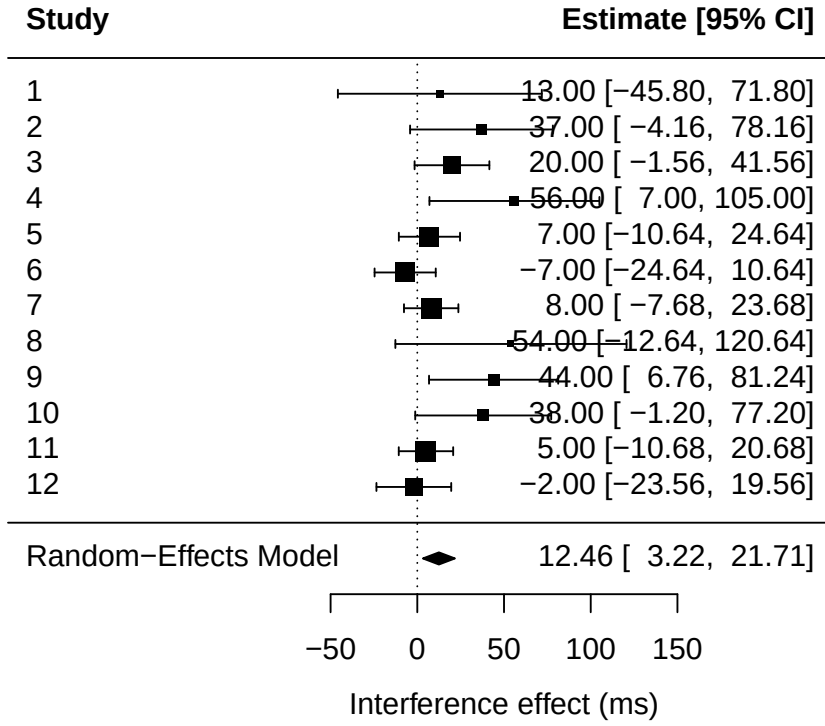
##
## Random-Effects Model (k = 12; tau^2 estimator: REML)
##
##   logLik  deviance      AIC      BIC     AICc
```

```
## -48.8404  97.6808 101.6808 102.4766 103.1808
##
## tau^2 (estimated amount of total heterogeneity): 83.5519 (SE = 100.7793)
## tau (square root of estimated tau^2 value):      9.1407
## I^2 (total heterogeneity / total variability):   35.41%
## H^2 (total variability / sampling variability):  1.55
##
## Test for Heterogeneity:
## Q(df = 11) = 18.0217, p-val = 0.0811
##
## Model Results:
##
## estimate      se      zval      pval      ci.lb      ci.ub
## 12.4600  4.7169  2.6416  0.0083  3.2150  21.7051  **
##
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The estimated effect is close to the Bayesian estimate, but the confidence interval is a bit tighter in the frequentist model (this difference is due to the fact that the Bayesian model's prior specification for the effect influences the posterior).

We can visualize the results of the frequentist meta-analysis using something called a forest plot (available in the `metafor` package):

```
forest(m,xlab="Interference effect (ms)")
```



2.5 *Aside: What happens if we set `sigma = TRUE` in `brms`?*

If we set `sigma = TRUE`, we won't be able to get estimates for ζ_n , since they are handled implicitly. The model presented formally in Equation 6 is equivalent to the following one in Equation 7. A critical difference is that ζ_n does not appear any more.

$$\begin{aligned}
 \text{effect}_n &\sim \text{Normal}(\zeta, \sqrt{\tau^2 + SE_n^2}) \\
 \zeta &\sim \text{Normal}(0, 100) \\
 \tau &\sim \text{Normal}_+(0, 100)
 \end{aligned} \tag{7}$$

This works because of the following property of normally distributed random variables:

If X and Y are two independent random variables, and

$$\begin{aligned}
 X &\sim \text{Normal}(\mu_X, \sigma_X) \\
 Y &\sim \text{Normal}(\mu_Y, \sigma_Y)
 \end{aligned} \tag{8}$$

then, Z , the sum of these two random variables is:

$$Z = X + Y \tag{9}$$

The distribution of Z has the following form:

$$Z \sim \text{Normal}\left(\mu_X + \mu_Y, \sqrt{\sigma_X^2 + \sigma_Y^2}\right) \quad (10)$$

In our case, let

$$\begin{aligned} U_n &\sim \text{Normal}(0, SE_n) \\ \zeta_n &\sim \text{Normal}(\zeta, \tau) \end{aligned} \quad (11)$$

Analogous to Equations 9 and 10, effect_n can be expressed as a sum of two independent random variables:

$$\text{effect}_n = U_n + \zeta_n \quad (12)$$

The distribution of effect_n will be

$$\text{effect}_n \sim \text{Normal}\left(\zeta, \sqrt{SE^2 + \tau^2}\right) \quad (13)$$

We can fit this in brms as follows. In this model specification, one should not include the $+(1 \mid \text{study_id})$, and the prior for τ should now be specified for sigma.

```
priors2 <- c(prior(normal(0, 100), class = Intercept),
             prior(normal(0, 100), class = sigma))
fit_sbi_sigma <- brm(effect | resp_se(`SE`, sigma = TRUE) ~ 1,
                     data = df_sbi,
                     prior = priors2,
                     control = list(
                       adapt_delta = .99,
                       max_treedepth = 10))
```

There are slight differences with `fit_sbi` due to the different parameterization and the sampling process, but the results are very similar:

```
posterior_summary(fit_sbi_sigma,
                  variable = c("b_Intercept", "sigma"))

##           Estimate Est.Error      Q2.5      Q97.5
## b_Intercept 13.42965   6.736934 2.6782374 28.99728
## sigma      12.07813   7.907773 0.6074978 30.33096
```

If we are not interested in the underlying effects in each study, this parameterization of the meta-analysis can be faster and more robust (i.e., it has less potential convergence issues). A major drawback is that we can no longer display a forest plot as we do in Figure 1.

2.6 Back to meta-analysis: Forest plots

Another interesting by-product of a random-effects meta-analysis is the possibility of displaying a forest plot (Figure 1). A forest plot shows the meta-analytic estimate (the parameter `b_Intercept` in `brms`) alongside the original estimates effect_n (and their SE_n) and the posterior distributions of the ζ_n for each study (we reconstruct these estimates by adding `b_Intercept` to the parameters starting with `r_` in `brms`). The original estimates are the ones fed to the model as data and the posterior distributions of the ζ_n are calculated, as in previous hierarchical models, after the information from all studies is pooled together. The ζ_n estimates are shrunk estimates of each study's (unknown) true effect, shrunk towards the grand mean ζ , and weighted by the standard error observed in each study n . The ζ_n for a particular study is shrunk more towards the grand mean ζ when the study's standard error is large (i.e., when the estimate is very imprecise). The code below shows how to build a forest plot step by step.

First, change the format of the data so that it looks like the output of `brms`:

```
df_sbi <- df_sbi %>%
  mutate(Q2.5 = effect - 2 * SE,
         Q97.5 = effect + 2 * SE,
         Estimate = effect,
         type = "original")
```

Extract the meta-analytical estimate:

```
df_Intercept <- posterior_summary(fit_sbi,
  variable = c("b_Intercept")
) %>%
  as.data.frame() %>%
  mutate(publication = "M.A. estimate", type = "")
```

For the pooled estimated effect (or fitted value) of the individual studies, we need the sum of the meta-analytical estimate (intercept) and each of the by-study adjustment. Obtain this with the `fitted()` function:

```
df_model <- fitted(fit_sbi) %>%
  # Convert matrix to data frame:
  as.data.frame() %>%
```

```
# Add a column to identify the estimates,
# and another column to identify the publication:
mutate(type = "adjusted",
        publication = df_sbi$publication)
```

Bind the observed effects, the meta-analytical estimate, and the fitted values of the studies together, and plot the data.

```
# the adjusted estimates and the meta-analysis estimate:
bind_rows(df_sbi, df_model, df_Intercept) %>%
# Plot:
ggplot(aes(x = Estimate,
            y = publication,
            xmin = Q2.5,
            xmax = Q97.5,
            color = type)) +
geom_point(position = position_dodge(.5)) +
geom_errorbarh(position = position_dodge(.5)) +
# Add the meta-analytic estimate and Credible Interval:
geom_vline(xintercept = df_Intercept$Q2.5,
            linetype = "dashed",
            alpha = .3) +
geom_vline(xintercept = df_Intercept$Q97.5,
            linetype = "dashed",
            alpha = .3) +
geom_vline(xintercept = df_Intercept$Estimate,
            linetype = "dashed",
            alpha = .5) +
scale_color_discrete(breaks = c("adjusted", "original"))

## Warning: 'geom_errobah()' was deprecated in ggplot2 4.0.0.
## i Please use the 'orientation' argument of 'geom_errorbar()' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

It is important to keep in mind that a meta-analysis is always going to yield biased estimates as long as we have publication bias: if a field has a tendency to allow only “big news” studies to be published, then the literature that will appear in the public domain will be biased, and any meta-analysis based on such information will be biased. Despite this limitation, a

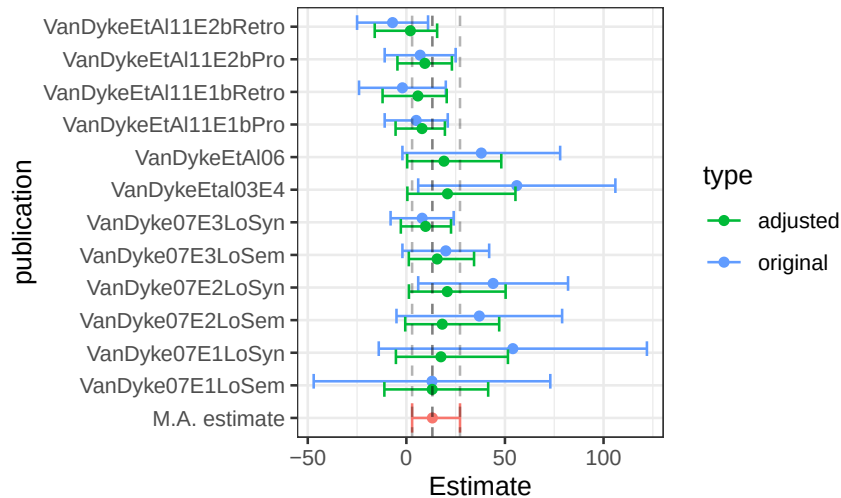


Figure 1: Forest plot showing the original and the adjusted estimates computed from each study from the random-effects meta-analysis. The error bars on the original estimates show 95% confidence intervals, and those on the adjusted estimates show 95% credible intervals.

meta-analysis is still a useful way to synthesize the known evidence; one just has to remember that the estimate from the meta-analysis is almost certain to be biased.

2.7 Stan version of the meta-analysis model

Even though brms can handle meta-analyses, fitting them in Stan allows us for more flexibility, which might be necessary in some cases. As a first attempt we could build a model that closely follows the formal specification given in equation 6.

```
data {
  int<lower=1> N;
  vector[N] effect;
  vector[N] SE;
  vector[N] study_id;
}
parameters {
  real zeta;
  real<lower = 0> tau;
  vector[N] zeta_n;
}
model {
  target += normal_lpdf(effect | zeta_n, SE);
  target += normal_lpdf(zeta_n | zeta, tau);
  target += normal_lpdf(zeta | 0, 100);
  target += normal_lpdf(tau | 0, 100);
}
```

```

    - normal_lccdf(0 | 0, 100);
  }

```

Fit the model as follows:

```

ma0 <- system.file("stan_models",
  "meta-analysis0.stan",
  package = "bcogsci"
)
ls_sbi <- list(N = nrow(df_sbi),
  effect = df_sbi$effect,
  SE = df_sbi$SE,
  study_id = df_sbi$study_id)
fit_sbi0 <- stan(ma0,
  data = ls_sbi,
  control = list(
    adapt_delta = .999,
    max_treedepth = 12))

```

```

## Warning: There were 17 divergent transitions after warmup. See
## https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup
## to find out why this is a problem and how to eliminate them.

```

```

## Warning: There were 3 chains where the estimated Bayesian Fraction of Missing Information was
## https://mc-stan.org/misc/warnings.html#bfmi-low

```

```

## Warning: Examine the pairs() plot to diagnose sampling problems

```

```

## Warning: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and medians
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#bulk-ess

```

```

## Warning: Tail Effective Samples Size (ESS) is too low, indicating posterior variances and tail
## Running the chains for more iterations may help. See
## https://mc-stan.org/misc/warnings.html#tail-ess

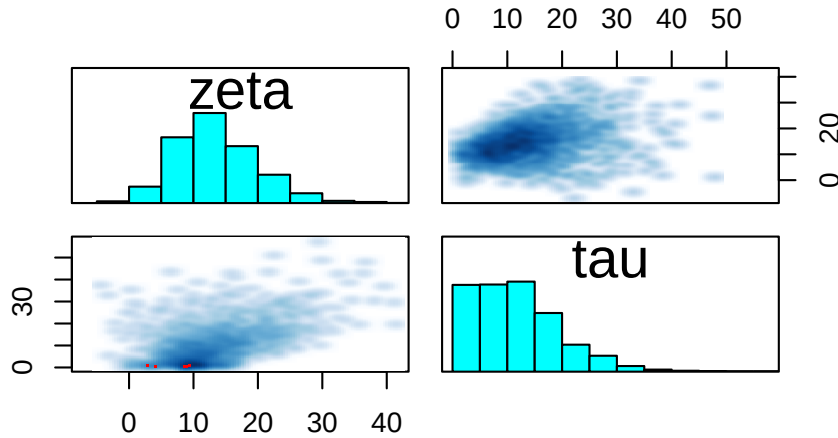
```

We see that there are warnings. As discussed before, we can use pairs plots to uncover pathologies in the sampling. Here we see the samples of zeta and tau are highly correlated:

```

pairs(fit_sbi0, pars = c("zeta", "tau"))

```



We face a similar problem as we faced earlier, namely, the sampler cannot properly explore the neck of the funnel-shaped space, because of the strong correlation between the parameters. The solution is a non-centered parameterization. Re-write Equation 6 as follows:

$$\begin{aligned}
 z_n &\sim \text{Normal}(0, 1) \\
 \zeta_n &= z_n \cdot \tau + \zeta \\
 \text{effect}_n &\sim \text{Normal}(\zeta_n, SE_n) \\
 \zeta &\sim \text{Normal}(0, 100) \\
 \tau &\sim \text{Normal}_+(0, 100)
 \end{aligned} \tag{14}$$

This works because if $X \sim \text{Normal}(a, b)$ and $Y \sim \text{Normal}(0, 1)$, then $X = a + Y \cdot b$.

Translate equation 14 into Stan code as follows in `meta-analysis1.stan`:

```

mal <- system.file("stan_models",
  "meta-analysis1.stan",
  package = "bcogsci")

fit_sb1 <- stan(mal,
  data = ls_sbi,
  control = list(adapt_delta = .999,
    max_treedepth = 12))

data {
  int<lower=1> N;
  vector[N] effect;
  vector[N] SE;
  vector[N] study_id;
}

```

```

parameters {
  real zeta;
  real<lower = 0> tau;
  vector[N] z;
}
transformed parameters {
  vector[N] zeta_n = z * tau + zeta;
}
model {
  target += normal_lpdf(effect| zeta_n, SE);
  target += std_normal_lpdf(z);
  target += normal_lpdf(zeta | 0, 100);
  target += normal_lpdf(tau | 0, 100)
    - normal_lccdf(0 | 0, 100);
}

```

The model converges with values virtually identical to the ones of the brms model.

```

print(fit_sbi1, pars = c("zeta", "tau"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##      mean se_mean   sd 2.5% 25%   50%   75% 97.5% n_eff Rhat
## zeta 13.38    0.18 6.14 2.89 9.23 12.77 16.96 27.55 1178    1
## tau  11.90    0.26 7.75 0.71 5.97 10.84 16.43 30.02  922    1
##
## Samples were drawn using NUTS(diag_e) at Sun Mar 29 09:33:28 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).

```

We can also reparameterize the model slightly differently, if we set $U_n \sim \text{Normal}(0, SE_n)$ then,

$$\text{effect}_n = U_n + \zeta_n \quad (15)$$

Then, given that $\zeta_n \sim \text{Normal}(\zeta, \tau)$,

$$\text{effect}_n \sim \text{Normal}(\zeta, \sqrt{SE^2 + \tau^2}) \quad (16)$$

See the aside above if it's not clear why this reparameterization works.

This is equivalent to the brms model where `sigma = TRUE`. As with brms, we lose the possibility of estimating the posterior of the true effect of the individual studies.

Write this in Stan as follows; this code is available in the file `meta-analysis2.stan` within the `bcogsci` package:

```
data {
  int<lower=1> N;
  vector[N] effect;
  vector[N] SE;
  vector[N] study_id;
}
parameters {
  real zeta;
  real<lower = 0> tau;
}
model {
  target += normal_lpdf(effect | zeta,
                        sqrt(square(SE) + square(tau)));
  target += normal_lpdf(zeta | 0, 100);
  target += normal_lpdf(tau | 0, 100)
    - normal_lccdf(0 | 0, 100);
}
```

Fit the model:

```
ma2 <- system.file("stan_models",
  "meta-analysis2.stan",
  package = "bcogsci"
)
fit_sbi2 <- stan(ma2,
  data = ls_sbi,
  control = list(adapt_delta = .9))

print(fit_sbi2, pars = c("zeta", "tau"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##      mean se_mean   sd 2.5% 25%   50%   75% 97.5% n_eff Rhat
## zeta 13.20    0.19 6.00 2.61 9.19 12.62 16.66 26.73 1048   1
## tau  11.48    0.22 7.54 0.72 5.73 10.32 15.97 29.40 1148   1
```

```
##
## Samples were drawn using NUTS(diag_e) at Sun Mar 29 09:33:48 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

This summary could be reported in an article by displaying the posterior means and 95% credible intervals of the parameters.

3 *Measurement-error models*

Measurement error models deal with the situation where some predictor or the dependent variable, or both, are observed with measurement error. This measurement error could arise because a variable is an average (i.e., its standard error can also be estimated), or because we know that our measurement is noisy due to limitations of our equipment (e.g., delays in the signal from the keyboard to the motherboard, impedance in the electrodes in an EEG system, etc.).

3.1 *Accounting for measurement error in individual differences in working memory capacity and reading fluency*

As a motivating example, consider the following data from Nicenboim et al 2018. For each subject, we have the partial-credit unit (PCU) scores of an operation span task as a measure of their working memory capacity along with their standard error. In addition, the reading fluency of each subject is calculated from a separate set of data based on the mean reading speeds (character/second) in a rapid automatized naming task [RAN]; the standard error of the reading speed is also available.

Of interest here is the extent of the association between working memory capacity (measured as PCU) and reading fluency (measured as reading speed in 50 characters per second). We avoid making any causal claims: It could be that our measure of working memory capacity really affects reading fluency or it could be the other way around. A third possibility is that there is a third variable (or several) that affects both reading fluency and working memory capacity. A treatment of

causality in Bayesian models can be found in chapters 5 and 6 of McElreath's book.

```
data("df_indiv")
df_indiv

## # A tibble: 100 x 5
##   subj mean_rspped se_rspped mean_pcu se_pcu
##   <dbl>      <dbl>      <dbl>      <dbl> <dbl>
## 1     1      0.0521  0.00113      0.738 0.0648
## 2     2      0.0479  0.00121      0.292 0.0315
## 3     3      0.0601  0.00117      0.408 0.0900
## 4     4      0.0522  0.000802     0.728 0.0756
## 5     5      0.0524  0.000764     0.449 0.0710
## 6     6      0.0493  0.00105     0.479 0.0633
## 7     7      0.0549  0.00195     0.726 0.104
## 8     8      0.0529  0.00145     0.486 0.0861
## 9     9      0.0451  0.00166     0.569 0.0933
## 10    10      0.0477  0.00115     0.5   0.0852
## # i 90 more rows
```

At first glance, we see a relationship between mean PCU scores and mean reading speed; see Figure 2. However, this relationship seems to be driven by two extreme data points on the top left corner of the plot.

```
df_indiv <- df_indiv %>%
  mutate(c_mean_pcu = mean_pcu - mean(mean_pcu))
ggplot(df_indiv, aes(x = c_mean_pcu, y = mean_rspped)) +
  geom_point() +
  geom_smooth(method = "lm")
```

A simple linear model shows a somewhat weak association between mean reading speed and centered mean PCU. The priors are relatively arbitrary but they are in the right order of magnitude given that reading speeds are quite short and well below 1.

```
df_indiv <- df_indiv %>%
  mutate(c_mean_pcu = mean_pcu - mean(mean_pcu))
priors <- c(
  prior(normal(0, 0.5), class = Intercept),
  prior(normal(0, 0.5), class = b),
  prior(normal(0, 0.5), class = sigma)
```

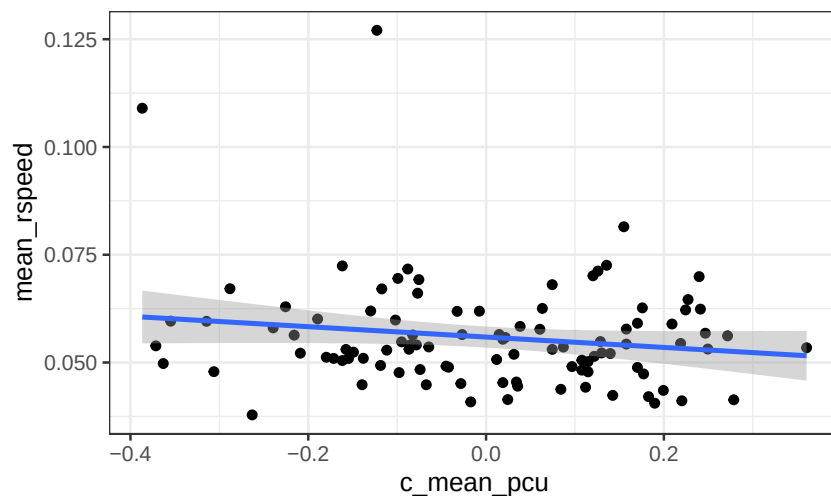


Figure 2: The relationship between (centered) mean PCU scores and mean reading speed.

```
)
fit_indiv <- brm(mean_rspped ~ c_mean_pcu,
  data = df_indiv,
  family = gaussian(),
  prior = priors
)

fit_indiv

## ...
## Population-Level Effects:
##           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept      0.06      0.00   0.05   0.06 1.00    5020    2941
## c_mean_pcu    -0.01      0.01  -0.03   0.00 1.00    2323    1879
##
## Family Specific Parameters:
##           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sigma      0.01      0.00   0.01   0.01 1.00    1882    1979
##
## ...

# Proportion of samples below zero
(Pb <- mean(as_draws_df(fit_indiv)$b_c_mean_pcu < 0))

## [1] 0.94025
```

The frequentist equivalent:

```

m_indiv_lm <- lm(mean_rspeed ~ c_mean_pcu,
  data = df_indiv)
summary(m_indiv_lm)

##
## Call:
## lm(formula = mean_rspeed ~ c_mean_pcu, data = df_indiv)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.021224 -0.007082 -0.002146  0.004357  0.069679
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.055903   0.001234  45.317   <2e-16 ***
## c_mean_pcu   -0.012028   0.007319  -1.643    0.104
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01234 on 98 degrees of freedom
## Multiple R-squared:  0.02682,    Adjusted R-squared:  0.01689
## F-statistic: 2.701 on 1 and 98 DF,  p-value: 0.1035

```

Figure 3 shows the posterior distribution of the slope in this model. Most of the probability mass is negative (94.025%), suggesting that a better PCU score is associated with slower reading speed rather than faster; that is, that a larger working memory capacity is associated with less reading fluency. This is not a very intuitive result and it could be the case that is driven by the two extreme data points. Rather than removing these data points, we'll examine what happens when the uncertainty of the measurements is taken into account.

```

mcmc_plot(fit_indiv,
  variable = "^b_c",
  regex = TRUE,
  type = "hist")

```

Taking this uncertainty of the measurement is important; in many practical research problems, researchers will often take average measurements like these and examine the correlation between them. However, each of those data points is being measured with some error (uncertainty), but this error

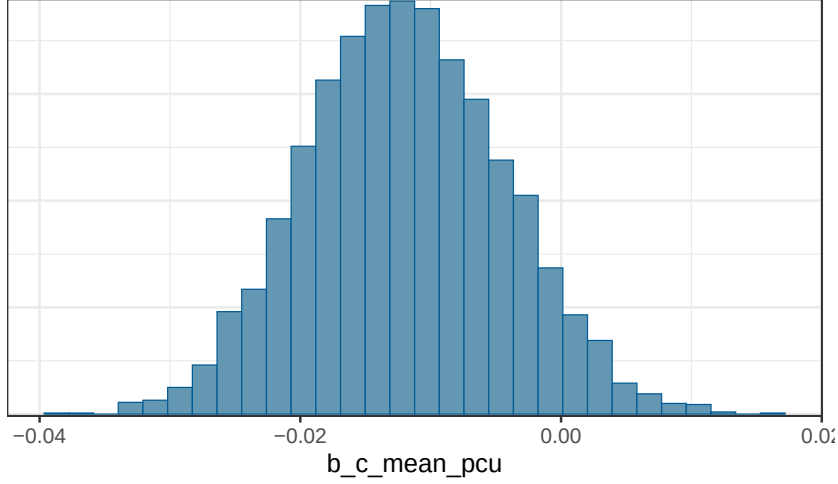


Figure 3: The posterior distribution of the slope in the linear model, modeling the effect of centered mean PCU on mean reading speed (the unit is 50 characters per second).

is being ignored when we take the averaged values. Ignoring this uncertainty leads to over-enthusiastic inferences. A measurement-error model solves this issue by taking uncertainty into account.

The measurement error model is stated as follows. There is assumed to be a true unobserved value $y_{n,TRUE}$ for the dependent variable, and a true unobserved value $x_{n,TRUE}$ for the predictor, where n is indexing the observation number. The observed values y_n and the predictor x_n are assumed to be generated with some error:

$$\begin{aligned} y_n &\sim \text{Normal}(y_{n,TRUE}, SE_y) \\ x_n &\sim \text{Normal}(x_{n,TRUE}, SE_x) \end{aligned} \quad (17)$$

The regression is fit to the (unknown) *true* values of the dependent and independent variables:

$$y_{n,TRUE} \sim \text{Normal}(\alpha + \beta x_{n,TRUE}, \sigma) \quad (18)$$

In addition, there is also an unknown standard deviation (standard error) of the latent unknown means that generate the underlying PCU means. I.e., we assume that each of the observed centered PCU scores are normally distributed with an underlying mean, χ , and a standard deviation τ . This is very similar to the meta-analysis situation we saw earlier: $\zeta_n \sim \text{Normal}(\zeta, \tau)$, where ζ_n was the true latent mean of each study, and ζ was the (unknown) true value of the parameter, and τ was the between-study variability.

$$x_{n,TRUE} \sim \text{Normal}(\chi, \tau) \quad (19)$$

The goal of the modeling is to obtain posterior distributions for the intercept and slope α and β (and the residual error standard deviation σ).

We need to decide on priors for all the parameters now. We use relatively vague priors, which can still be considered regularizing priors based on our knowledge of the order of magnitude of the measurements. In situations where not much is known about a research question, one could use such vague priors.

$$\begin{aligned} \alpha &\sim \text{Normal}(0, 0.5) \\ \beta &\sim \text{Normal}(0, 0.5) \\ \chi &\sim \text{Normal}(0, 0.5) \\ \sigma &\sim \text{Normal}_+(0, 0.5) \\ \tau &\sim \text{Normal}_+(0, 0.5) \end{aligned} \quad (20)$$

3.2 The brms version of the measurement error model

In brms, the model specification would be as follows:

```
priors_me <- c(
  prior(normal(0, 0.5), class = Intercept),
  prior(normal(0, 0.5), class = b),
  prior(normal(0, 0.5), class = meanme),
  prior(normal(0, 0.5), class = sdme),
  prior(normal(0, 0.5), class = sigma)
)
```

Here the parameter with class meanme and sdme refer to the unknown mean and standard deviation (standard error) of the latent unknown means that generate the underlying PCU means, χ and τ in 20. Once we decide on the priors, we use `resp_se(.)` with `sigma = TRUE` (i.e, we don't estimate $y_{n,TRUE}$ explicitly) and we use `me(c_mean_pcu, se_pcu)` to indicate that the dependent variable `c_mean_pcu` is measured with error and `se_pcu` is its SE.

```
fit_indiv_me <- brm(mean_rspeed | resp_se(se_rspeed, sigma = TRUE) ~
  me(c_mean_pcu, se_pcu),
  data = df_indiv,
```

```

    family = gaussian(),
    prior = priors_me)

short_summary(fit_indiv_me)

## ...
## Population-Level Effects:
##               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept           0.05      0.00    0.05    0.06 1.00     4232     2811
## mec_mean_pcuse_pcu   -0.00      0.01   -0.01    0.01 1.00     7312     3304
##
## Family Specific Parameters:
##               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sigma           0.01      0.00    0.01    0.01 1.00     7844     2629
##
## ...

# Proportion of samples below zero
# Parameter names can be found out with `variables(fit_indiv_me)`
(Pb_me <- mean(as_draws_df(fit_indiv_me)$bsp_mec_mean_pcuse_pcu < 0))

## [1] 0.6215

```

The posterior for the slope is plotted in Figure 4; this figure shows that the association between PCU scores and reading speed is much weaker once measurement error is taken into account: The posterior is much more uncertain (much more widely distributed) than in the simple linear model we fit above (compare Figure 4 with Figure 3), and the direction of the association is now unclear, with 62% of the probability mass below zero, rather than 94%.

Figure 5 visualizes the main reason why we have no clear association in the measurement error analysis: the two points at the top left part of the plot that were driving the effect have very large SE for the measurement of reading speed. The code to produce it appears below and overlays several (250) regression lines that correspond to different samples of the posterior distribution with the measurements of reading speed and PCU.

```

df_reg <- as_draws_df(fit_indiv_me) %>%
  dplyr::select(alpha = b_Intercept, beta = bsp_mec_mean_pcuse_pcu) %>%
  slice(1:250)
ggplot(df_indiv, aes(x = c_mean_pcu, y = mean_rspeed)) +
  geom_point() +

```

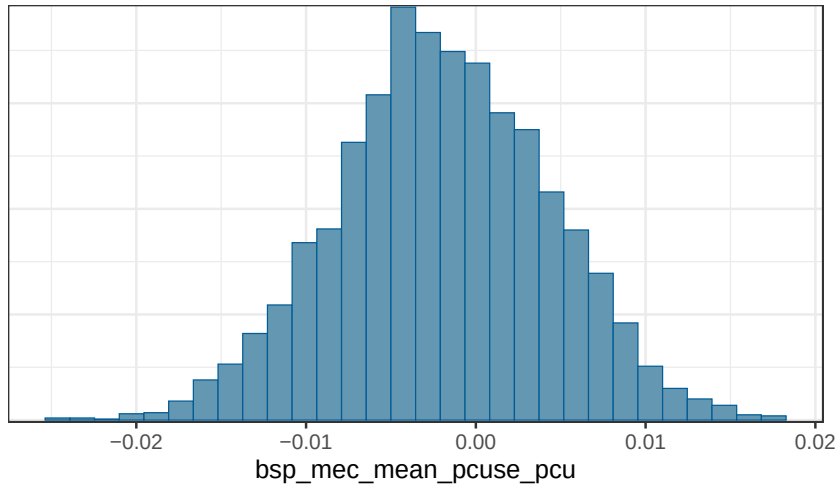


Figure 4: The posterior distribution of the slope in the measurement error model, modeling the effect of centered mean PCU on mean reading speed.

```
geom_errorbarh(aes(xmin = c_mean_pcu - 2 * se_pcu,
                  xmax = c_mean_pcu + 2 * se_pcu),
              alpha = .5, linetype = "dotted") +
geom_errorbar(aes(ymin = mean_rspeed - 2 * se_rspeed,
                 ymax = mean_rspeed + 2 * se_rspeed),
              alpha = .5, linetype = "dotted") +
geom_abline(aes(intercept = alpha, slope = beta),
            data = df_reg,
            alpha = .05)
```

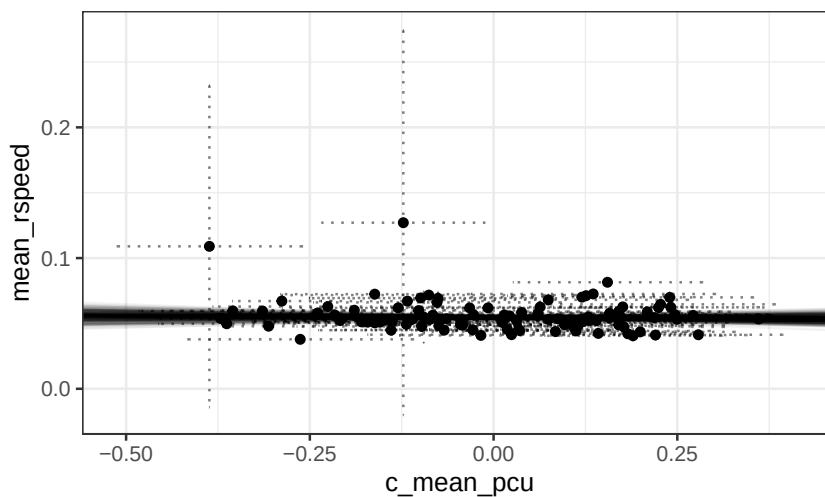


Figure 5: The relationship between centered mean PCU scores and mean reading speed accounting for measurement error. The error bars represent two standard errors. The regression lines are produced with 250 samples of the intercept and slope from the posterior distribution.

Of course, the conclusion here cannot be that there is no association between PCU scores and reading speed. In order

to claim an absence of an effect, we would need to use Bayes factor or cross validation.

3.3 The Stan version of the measurement error model

As it happened when we modeled the meta-analysis, the main difficulty for modeling measurement error models directly in Stan is that we need to reparameterize the models to avoid correlations between samples of different parameters. The two changes that we need to do to the parameterization of our model presented in Equation 20 are the following.

1. Sample from an auxiliary parameter z_n rather than directly from $x_{n,TRUE}$, as we did in Equation 14:

$$\begin{aligned} z_n &\sim \text{Normal}(0, 1) \\ x_{n,TRUE} &= z_n \cdot \tau + \chi \\ x_n &\sim \text{Normal}(x_{n,TRUE}, SE_x) \end{aligned} \quad (21)$$

2. Don't model $y_{n,TRUE}$ explicitly; rather take into account the SE and the variation on $y_{n,TRUE}$ in the following way:

$$y_n \sim \text{Normal}\left(\alpha + \beta x_{n,TRUE}, \sqrt{SE_y^2 + \sigma^2}\right) \quad (22)$$

We are now ready to write this in Stan; the code is in the model called `me.stan`:

```
data {
  int<lower=1> N;
  vector[N] x;
  vector[N] SE_x;
  vector[N] y;
  vector[N] SE_y;
}
parameters {
  real alpha;
  real beta;
  real chi;
  real<lower = 0> sigma;
  real<lower = 0> tau;
  vector[N] z;
}
```

```

transformed parameters {
  vector[N] x_true = z * tau + chi;
}
model {
  target += normal_lpdf(x | x_true, SE_x);
  target += normal_lpdf(y | alpha + beta * x_true,
                        sqrt(square(SE_y) + square(sigma)));
  target += std_normal_lpdf(z);
  target += normal_lpdf(alpha | 0, 0.5);
  target += normal_lpdf(beta | 0, 0.5);
  target += normal_lpdf(chi | 0, 0.5);
  target += normal_lpdf(sigma | 0, 0.5)
    - normal_lccdf(0 | 0, 0.5);
  target += normal_lpdf(tau | 0, 0.5)
    - normal_lccdf(0 | 0, 0.5);
}

```

Fit the model:

```

me <- system.file("stan_models",
  "me.stan", package = "bcogsci")
ls_me <- list(N = nrow(df_indiv),
  y = df_indiv$mean_rspeed,
  SE_y = df_indiv$se_rspeed,
  x = df_indiv$c_mean_pcu,
  SE_x = df_indiv$se_pcu)
fit_indiv_me_stan <- stan(me, data = ls_me)

print(fit_indiv_me_stan, pars = c("alpha", "beta", "sigma"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##      mean se_mean   sd  2.5%   25%   50%   75%  97.5% n_eff Rhat
## alpha 0.05      0 0.00  0.05  0.05 0.05 0.06  0.06  4282   1
## beta  0.00      0 0.01 -0.01 -0.01 0.00 0.00  0.01  5569   1
## sigma 0.01      0 0.00  0.01  0.01 0.01 0.01  0.01  5668   1
##
## Samples were drawn using NUTS(diag_e) at Sun Mar 29 09:34:59 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).

```

The posterior distributions are similar to those that we obtained with brms.